Patrones de diseño verificados en FaceLit

Fecha de revisión de referencias: 4 de noviembre de 2026.

Este documento rectifica la clasificación de patrones mediante inspección del código local. Complementa la sección 6.1 del documento general y la sección de diseño del artículo. Las propuestas del anexo `Introducción a framework.docx` se contrastan como material de referencia, sin asumir que ya estén implementadas.

Alcance y criterio

Se revisaron `FaceLit`, `FaceLit-Backend`, `micro-facial` y `FaceLit-DB`. Las referencias son relativas a este archivo y apuntan a los repositorios hermanos del espacio de trabajo; necesitan esa misma disposición para abrirse. Se indican archivos y símbolos para poder localizarlos también en cada repositorio por separado.

Repositorio	Rama y HEAD consultados	Alcance
FaceLit	`main` · `6efd92c`	Pantallas, hooks, contextos, stores y servicios compartidos.
FaceLit-Backend	`main` · `2ba7398`	Controladores, servicios, repositorios, DTO, seguridad, eventos y cliente facial.
micro-facial	`main` · `166e4a7`	Rutas FastAPI, servicio, repositorio, sesiones y trabajador facial.
FaceLit-DB	`main` · `cd069ae`	Changelogs, migraciones e integridad de datos.

Los hashes identifican los `HEAD` locales consultados el 4 de noviembre en las ramas `main`; no acreditan por sí solos qué revisión está desplegada. Se volvieron a cotejar los archivos enlazados y los ejemplos de código contra estos árboles. Es una revisión estática, sin ejecución de la aplicación ni validación del despliegue. Una ausencia de evidencia se expresa como «no verificado», no como prueba absoluta de inexistencia.

Revisión del estado del proyecto al 4 de noviembre

Los archivos y mecanismos citados se cotejaron en los árboles `main` indicados arriba. La revisión no constituye una auditoría completa de patrones ni de comportamiento en ejecución.

La arquitectura de ejecución se documenta como cliente Expo → API Spring Boot → llamada HTTP interna a FastAPI → PostgreSQL con pgvector. El backend y el servicio facial comparten PostgreSQL; esta separación de responsabilidades no convierte automáticamente el sistema en microservicios autónomos ni en una arquitectura distribuida dirigida por eventos. Los eventos Spring citados son locales al proceso. El despliegue reportado usa Hostinger y Caddy, con PostgreSQL, Spring y FastAPI en red interna.

El flujo facial vigente usa los retos `MOVE_UP`, `MOVE_DOWN`, `MOVE_CLOSER` y `MOVE_AWAY`; la secuencia documentada pide dos retos distintos con siete imágenes por reto. El umbral observado `0.65` es de similitud, no una medida de precisión. Estos datos describen el flujo y sus límites, no patrones de diseño adicionales.

Para diseño funcional, consulte Flujos Funcionales. Para la propuesta consolidada, consulte Diseño de Software. El equipo confirma que login, CSV, SMTP, cámara y asistencia funcionan. La prueba de restauración y la política de datos biométricos siguen siendo tareas operativas/documentales, no defectos funcionales reportados.

Resultado por repositorio

Repositorio	Patrones y prácticas confirmados	Clasificación precisa
FaceLit-Backend	Repository, servicios como fachada, Observer mediante eventos Spring, Builder de bibliotecas, fábricas estáticas, adaptación de tipos y cadena de filtros.	Combina patrones de diseño, patrones de aplicación y mecanismos del framework; no todos son patrones GoF propios.
FaceLit	Observer en stores y eventos; Context/Provider, hooks reutilizables, composición y Proxy de acceso remoto mediante servicios HTTP.	Observer tiene implementación explícita; Proxy describe el papel de intermediación de los servicios HTTP; Context, hooks y composición son prácticas de React.
micro-facial	Repository, Service Layer, inyección de sesión, encapsulación del motor facial y carga diferida con caché.	No se verificó una familia de estrategias ni de adaptadores faciales intercambiables.
FaceLit-DB	Migraciones versionadas, rollback e integridad relacional.	Son prácticas de evolución y diseño de datos; este repositorio no demuestra por sí mismo patrones GoF.

1. Backend Java

Repository y Service Layer

ProgramRepository extiende `JpaRepository<Program, UUID>` y concentra operaciones de persistencia. ProgramServiceImpl recibe repositorios por constructor, valida y coordina operaciones transaccionales. ProgramController delega en el servicio.

El flujo observado es controlador → servicio → repositorio → base de datos. Repository separa el acceso a datos; no garantiza por sí solo que nadie pueda borrar registros: esa protección depende de permisos, reglas, restricciones y transacciones.

Facade: servicios que simplifican casos de uso

`ProgramServiceImpl` ofrece operaciones como `create` y `findAll` y coordina repositorios y bitácora. Puede describirse como una fachada de aplicación sobre esas operaciones internas. La evidencia es su coordinación real, no únicamente la anotación `@Service`. La categoría principal de estos componentes sigue siendo Service Layer; no se debe convertir automáticamente todo servicio en una implementación estricta de Facade GoF.

Observer / publicación y suscripción de eventos

NotificationServiceImpl publica `EmailDispatchRequested` mediante `ApplicationEventPublisher`. EmailDispatchServiceImpl, en `onEmailDispatchRequested`, recibe el evento con `@TransactionalEventListener(AFTER_COMMIT)` y `@Async`.

Existe otro ejemplo: AccountAccessService publica `InitialCredentialsEmailRequested` y InitialCredentialEmailListener lo escucha después del commit.

Esto demuestra publicación/suscripción dentro del proceso Spring. No demuestra un broker distribuido ni que todo el reconocimiento facial se ejecute como cuatro observadores simultáneos. La verificación facial también tiene un flujo HTTP explícito en FacialEmbeddingVerificationClient.

Fábricas estáticas de DTO

Se verificaron `LoginResponseDTO.success(...)`, `AssignRoleResponseDTO.success(...)` y `UserConfigurationResponseDTO.created(...)` / `updated(...)`:

- LoginResponseDTO.
- AssignRoleResponseDTO.
- UserConfigurationResponseDTO.

Construyen respuestas con inicialización uniforme. El nombre correcto aquí es **métodos de fábrica estáticos**. No se verificó la jerarquía de creadores con método redefinido por subclases que caracteriza Factory Method GoF, ni una fábrica de personas por rol como la propuesta en el anexo.

Builder proporcionado por bibliotecas

JwtServiceImpl utiliza `Jwts.builder()` para construir tokens por etapas. `FacialEmbeddingVerificationClient` utiliza `RestClient.builder()` para configurar su cliente HTTP. Son usos verificables de APIs Builder externas. Getters, setters y constructores generados por Lombok no constituyen por sí solos Builder; no se encontró `@Builder` en las fuentes Java revisadas.

Adaptación de tipos: AttributeConverter

InstructorTypeConverter implementa `AttributeConverter<InstructorType, String>` y traduce `ESPECIFICO` a `SPECIFIC` y `TRANSVERSAL` a `CROSS-CUTTING`, con conversión inversa. Es una adaptación concreta al contrato de persistencia, descrita con mayor precisión como **convertidor JPA**.

Los DTO transportan datos del contrato de entrada/salida; su existencia por sí sola no demuestra Adapter GoF. El cliente HTTP facial también encapsula una integración externa, sin demostrar una familia intercambiable de adaptadores OpenCV/FaceNet/DeepFace.

Singleton gestionado e inyección de dependencias

`ProgramServiceImpl`, `JwtServiceImpl` y otros componentes son beans de Spring y reciben dependencias por constructor. Sin un scope explícito distinto, Spring los gestiona con alcance singleton por contenedor y definición de bean. Esto es alcance y ciclo de vida del framework, no una clase Singleton GoF con constructor privado e instancia global controlada por el proyecto. Tampoco significa una sola instancia para todos los procesos o servidores.

Cadena de filtros e interceptores

JwtFilter extiende `OncePerRequestFilter` y delega mediante `filterChain.doFilter(...)`. SecurityConfig integra ese filtro y `RecoveryRateLimitFilter` en la cadena. Se puede documentar como cadena de responsabilidad provista por Servlet/Spring Security, con filtros de seguridad propios. `JwtFilter` no implementa por sí mismo Proxy GoF.

RealtimeChangeInterceptor implementa `HandlerInterceptor` y publica cambios tras solicitudes de modificación exitosas bajo sus condiciones. Es otro mecanismo de intercepción; no equivale al evento transaccional de correo.

Auditoría compartida

AuditBase utiliza `@MappedSuperclass` y campos comunes de auditoría. Es reutilización por herencia y mapeo JPA. No se verificó ahí un algoritmo plantilla con pasos redefinibles que justifique Template Method.

2. Frontend React Native / Expo

Observer explícito en stores y eventos

academicStore.ts mantiene un `Set` de listeners, permite suscribirse/desuscribirse y llama a `emit()` cuando cambia el estado. useAcademic.ts lo consume con `useSyncExternalStore`. También existen stores para notificaciones, ambientes y registro facial.

sessionEvents.ts ofrece `onSessionRejected` y `notifySessionRejected`. realtime.ts permite `subscribeRealtime` y notifica a los suscriptores al recibir mensajes WebSocket. Estos son ejemplos directos de Observer/publicación-suscripción mediante funciones.

Context/Provider, hooks y composición

AuthContext, ThemeContext e I18nContext distribuyen estado y operaciones mediante Context/Provider. Los hooks de las funcionalidades encapsulan lógica reutilizable y los componentes compartidos permiten composición de interfaz. Son patrones y prácticas propios de React; no prueban por sí solos MVC, MVVM ni Singleton GoF.

Proxy de acceso remoto, cliente HTTP e interceptores

authService.js representa operaciones remotas mediante funciones locales como `login`, `getMyProfile` y `logout`, que delegan en endpoints del backend. Los componentes consumen esas funciones sin construir cada solicitud HTTP. En este sentido, la capa de servicios cumple el papel de **Proxy de acceso remoto**: intermedia el acceso a capacidades del servidor y encapsula su comunicación. Esta clasificación describe su responsabilidad funcional; no se verificó una estructura GoF estricta con proxy y objeto real implementando una misma interfaz explícita.

api.js proporciona el cliente Axios común y configura interceptores que agregan el token y gestionan respuestas de sesión rechazada. Estos interceptores complementan la intermediación; por sí solos no demuestran Proxy GoF. Una instancia exportada por un módulo se comparte dentro de su entorno de ejecución y no garantiza una instancia única entre dispositivos. El Proxy aquí descrito tampoco equivale al proxy inverso de infraestructura mencionado en documentos de despliegue.

3. Microservicio facial Python

Repository, Service Layer y dependencias

facial_embeddings.py obtiene la sesión con `Depends(db_session)` y llama a `FacialEmbeddingService`.

facial_embedding_service.py coordina validaciones, generación de embeddings y persistencia. FacialEmbeddingRepository encapsula consultas SQL. El servicio realiza commit/rollback en sus operaciones de escritura.

session.py define el engine y `async_sessionmaker`, y entrega sesiones con un generador asíncrono. La sesión de cada solicitud no debe describirse como una única sesión global compartida.

Encapsulación del motor, carga diferida y caché

FaceEmbeddingWorker encapsula InsightFace y devuelve `GeneratedEmbedding`. `_get_model()` carga el modelo cuando se necesita; `get_face_embedding_worker()` usa `@lru_cache` para reutilizar el resultado de la función sin argumentos. Es carga diferida y caché por proceso, no garantía de una única instancia en todos los workers o servidores, ni prohibición de construir otras instancias.

Esta encapsulación permite describir un envoltorio del motor externo. No se verificaron clases `OpenCVAdapter`, `FaceNetAdapter` o `DeepFaceAdapter`, un contrato común de adaptadores ni estrategias intercambiables por jornada. Los condicionales de desafíos de vida no bastan para afirmar Strategy GoF.

4. Repositorio de base de datos

changelog-master.yaml ordena los changelogs DDL, DML, DCL y TCL. Los changesets de índices incluyen rollback. El índice de asistencia facial aporta integridad frente a registros duplicados bajo sus condiciones.

Esto es evolución versionada e integridad de datos. Repository pertenece al código Java/Python que accede a la base, no al repositorio de migraciones por llamarse «repositorio».

Rectificaciones de la documentación previa

Afirmación o etiqueta	Rectificación basada en código
«Factory Method» para `success`, `created` y `updated`, en el documento general, sección 6.1.	Cambiar el encabezado a «Fábricas estáticas de DTO». El párrafo ya reconoce que no es Factory Method GoF estricto.
«Los DTOs son Adapter».	Documentarlos como contratos de transferencia de datos; usar `InstructorTypeConverter` como evidencia concreta de adaptación de tipos JPA.
«Proxy» para `JwtFilter`.	Usar «Filtro de seguridad integrado en una cadena». El documento general y el artículo ya matizan que no es Proxy GoF puro; conservar esa precisión.
Lista de patrones del backend sin Repository ni Observer en la sección 6.1.	Incorporar Repository y eventos Spring con los ejemplos anteriores.
Factory de aprendiz, instructor y administrador propuesta en el anexo.	No verificada como familia de creadores en el código revisado. Mantener como propuesta.
Strategy de reglas por jornada, propuesta en el anexo.	No verificada una interfaz y estrategias intercambiables de ese tipo. Mantener como propuesta.
Cuatro observadores de asistencia de Raspberry, propuestos en el anexo.	No afirmar ese flujo como implementado. Sí existen eventos de correo y suscripciones del frontend verificables.
Adaptadores OpenCV/FaceNet/DeepFace propuestos en el anexo.	No verificados. El trabajador actual encapsula InsightFace y utilidades de procesamiento.
«Repository impide borrados accidentales».	La separación de acceso a datos facilita el control; las garantías necesitan reglas, permisos e integridad de base de datos.
Singleton como una sola instancia para todo el sistema.	Precisar el ámbito: contenedor Spring, módulo del frontend o caché del proceso Python, según corresponda.
Herencia de auditoría como Template Method.	Describir `AuditBase` como herencia y mapeo JPA; el documento general ya hace esa distinción.

Arquitectura: clasificación separada

El backend Java está organizado por módulos funcionales y capas y se ejecuta como una aplicación Spring Boot. Convive con el servicio facial FastAPI, el frontend y PostgreSQL. Esto permite describir un backend monolítico modular con servicio facial separado, dentro de un sistema cliente-servidor. Sus eventos locales y WebSocket no bastan para clasificar todo el

sistema como arquitectura distribuida dirigida por eventos.

La organización por carpetas y la existencia de interfaces tampoco acreditan por sí solas DDD completo, arquitectura hexagonal, Clean Architecture, MVC o MVVM estrictos. RBAC es un modelo de control de acceso; REST y WebSocket son mecanismos de comunicación. Deben explicarse aparte del catálogo de patrones de diseño.

Redacción corregida para el proyecto

> FaceLit aplica Repository y Service Layer en el backend Java y el microservicio Python. Los servicios Java ofrecen fachadas de casos de uso y utilizan inyección de dependencias y beans con alcance singleton gestionado por Spring. La publicación y escucha de eventos transaccionales de correo, junto con los stores y suscripciones del frontend, evidencian Observer/publicación-suscripción. Se utilizan fábricas estáticas para DTO de respuesta, APIs Builder de bibliotecas, conversión de tipos JPA y filtros integrados en la cadena de seguridad. El frontend emplea Context/Provider, hooks, composición y una capa de servicios HTTP que cumple el papel de Proxy de acceso remoto. El microservicio encapsula el motor facial con carga diferida y caché por proceso, y la base de datos evoluciona mediante migraciones versionadas. Las propuestas de estrategias por jornada, fábricas de personas y adaptadores faciales intercambiables no se consideran implementadas sin evidencia adicional.